

ConnectX 완전정보 게임 AI 전략 비교 분석

탐색 기반 vs 딥러닝 기반 알고리즘의
성능 평가 및 최적 전략 도출 연구



팀명

Connect 4 (4조)



팀원

A72066 이가영, A72076 이현지



기간

2025-02

github.com/gayeongge/2025_RL_Project

Table of Contents

ConnectX 완전정보 게임
AI 전략 비교 분석 프로젝트의
전체 발표 순서입니다.

🕒 예상 발표 시간: 15분

01 프로젝트 개요 연구 주제, ConnectX 규칙, 환경 특성

02 탐색 알고리즘 One/N-Step, MCTS, MTD(f) 구조 및 특징

03 학습 알고리즘 DQN, Double DQN, Dueling DQN 설계

04 실험 설계 및 결과 승률, 강제수 대응, 게임 길이, 탐색 비용 분석

05 결론 및 고찰 핵심 발견사항 요약 및 향후 연구 계획

프로젝트 개요 및 환경

🔬 연구 주제

완전정보 게임(Perfect Information Game)인 ConnectX 환경에서, 전통적인 탐색 기반 알고리즘(Search-Based)과 현대적인 딥러닝 기반 강화학습(Deep RL)의 성능과 효율성을 비교 분석함

🎯 연구 목표

각 알고리즘의 승률, 강제수 대응 능력, 의사결정 시간 등을 정량적으로 측정하여, 제한된 자원(시간/연산) 하에서의 최적 AI 전략을 도출함

One-Step

N-Step

MCTS

MTD(f)

DQN

💻 실험 환경

Kaggle ConnectX 시뮬레이터 활용.
에이전트는 현재 보드 상태를 완전히 관측(Fully Observable)하며,
행동 결과는 결정적(Deterministic)임

🎮 ConnectX 게임 규칙



보드 크기

6행(Rows) x 7열(Columns) 격자형 보드 사용



승리 조건

가로, 세로, 대각선 방향으로 자신의 돌 4개를 연속으로 연결하면 즉시 승리



중력 (Gravity)

플레이어는 열(Column)만 선택 가능하며, 돌은 해당 열의 가장 아래 빈 칸으로 떨어짐



진행 방식

2인 턴제 게임 (Turn-based), 선공(Player 1)과 후공(Player 2) 교대



Connect 4의 확장형으로, 4목(4-in-a-row) 규칙을 따릅니다.

01 탐색 알고리즘 Search-Based



One-Step Lookahead

바로 다음 수만 평가하는
Greedy 방식의 초고속 정책



N-Step Lookahead

Minimax 알고리즘을 적용하여
3수 앞(Ply 3)까지 깊이 탐색



MCTS + Heuristic

몬테카를로 트리 탐색과
위험 감지 휴리스틱의 결합



MTD(f) + Negamax

메모라이제이션(TT)과 반복 심화로
최적화된 고성능 탐색

One-Step Lookahead 알고리즘

재귀적 탐색을 통한 미래 상태 예측 및 최적 수 결정

- 아이디어: 바로 다음 상태만 평가해 가장 높은 휴리스틱 값을 고른다.
- 복잡도: $O(\text{branching factor } B)$; 매우 낮은 지연(latency).
- 장점: 연산 속도가 매우 빠름 초고속, 휴리스틱 디버깅·튜닝이 쉽고 결정적.
- 단점: 지연 보상·함정에 취약, 휴리스틱 편향에 민감.
- 주 용도: 베이스라인/롤아웃 정책, 큰 분기에서 빠른 근사, 탐색 초기 시드.
- 한계: '근시안적(Myopic)'임. 당장의 3목을 막느라 다음 턴에 상대에게 4목을 내어주는 수(Trap)를 파악하지 못함.

One-Step Lookahead State,Action,Reward 설계

State, Action, Reward 구조 심층 분석

State (상태 표현)

게임 환경(Environment)에서 제공하는 1차원 리스트 형태의 보드 데이터를 분석하기 쉬운 (rows, columns) 형태의 2차원 배열로 변환.

Action (행동 선택)

유효한 모든 열(Column)에 대해 가상의 토큰을 떨어뜨려보는 시뮬레이션을 수행.
각 행동에 대한 heuristic 점수를 계산한 후, 가장 높은 점수를 가진 행동을 선택.
만약 최고 점수가 여러 개라면 무작위(Random)로 하나를 선택하여 다양성을 확보.

가능한 action : 첫번째 열 ~ 여섯번째 열을 선택

Reward (보상 체계)

학습된 보상이 아닌, PATTERN_WEIGHTS를 활용한 순수 휴리스틱 평가.
evaluate_board 함수가 가능한 모든 4칸 윈도우를 스캔하여 내 4/3/2목과 상대의 위협을 +/- 가중치로 합산해 현재 수의 가치를 평가.

핵심 포인트 요약

Data Transformation 1차원 Raw Data를 2차원 Grid로 변환하여 공간적 패턴 인식을 최적화함

Decision Strategy Greedy(탐욕적) 접근법을 사용하여 즉각적인 이득을 최대화하며, Tie-break 시 무작위성을 부여하여 고착화 방지

Evaluation Metric 승리(4목), 공격(3목), 방어(상대 3목)에 대해 사전 정의된 가중치를 적용하여 정량적 평가 수행

Computational Cost 단 1수 앞만 내다보므로 연산 비용이 매우 낮아(< 5ms) 실시간 대응에 유리함

One-Step Lookahead Greedy Strategy - 가중치 정의

휴리스틱 가중치 기반의 즉시 보상 최대화 전략

💡 핵심 개념

현재 상태에서 가능한 모든 열(Column)에 대해 딱 한 수 앞 (t+1)만 시뮬레이션하고, 미리 정의된 패턴 가중치 점수가 가장 높은 곳을 선택.

⚙️ 패턴별 가중치 (Hyperparameters)

my_4	내 4목 완성 (승리)	+1,000,000
my_3	내 3목 연결 (공격)	+1,000
my_2	내 2목 연결 (포석)	+10
opp_3	상대 3목 저지 (방어)	-50,000
opp_2	상대 2목 견제	-100

방어 우선: 상대 3목 페널티(-50k)가 내 3목 보상(+1k)보다 50배 큼.

submission_simple_greedy_baseline.py

패턴별 가중치

```
PATTERN_WEIGHTS = {  
    "my_4": 1_000_000.0,    # 내가 4개 연결  
    "my_3": 1_000.0,        # 내가 3개 연결  
    "my_2": 10.0,           # 내가 2개 연결  
    "opp_3": -50_000.0,     # 상대 3개 연결  
    "opp_2": -100.0,        # 상대 2개 연결  
}
```

One-Step Lookahead Greedy Strategy - 상태 가치 평가

보드판의 현재 상태가 얼마나 유리한지 수치로 계산하여 최적 선택

☞ 상태 가치 함수 (State Value Function)

강화학습의 Value Function $V(s)$
와 유사하게, 현재 보드 상태가 얼마나 유리한지를 수치적으로 평가.

보드에 존재하는 5가지 핵심 패턴(내 4목, 상대 3목 등)의 개수(Feature)와 미리 설정된 가중치(Weight)의 선형 결합(Linear Combination)으로 점수를 계산하여 즉각적인 보상을 예측.

$$V(s) = \sum_i (w_i \times \text{count}(\text{pattern}_i))$$

submission_simple_greedy_baseline.py

```
def evaluate_board(grid, me, cfg):
    opponent = 1 if me == 2 else 2
    my_two = count_pattern(grid, 2, me, cfg)
    my_three = count_pattern(grid, 3, me, cfg)
    my_four = count_pattern(grid, 4, me, cfg)
    opp_two = count_pattern(grid, 2, opponent, cfg)
    opp_three = count_pattern(grid, 3, opponent, cfg)
    w = PATTERN_WEIGHTS
    score = (
        w["my_4"] * my_four +
        w["my_3"] * my_three +
        w["my_2"] * my_two +
        w["opp_2"] * opp_two +
        w["opp_3"] * opp_three
    )
    return float(score)
```


One-Step Lookahead Greedy Strategy - 에이전트 의사 결정

보드판의 현재 상태가 얼마나 유리한지 수치로 계산하여 최적 선택

탐욕적 정책 (Greedy Policy)

현재 상태에서 딱 한수 앞 (t+1)만 내다보고, 다음 상태의 가치 $V(s')$ 가 최대가 되는 행동을 선택.

최적해가 여러개일 경우 무작위로 선택 (Tie-break)하여 결정론적 패턴에 빠지는 것을 방지하고 약간의 탐험 효과를 줌.

$$\pi(s) = \operatorname{argmax}_a V(s' | s, a)$$

submission_simple_greedy_baseline.py

```
def my_agent(obs, config):
    # (생략)
    # 1. 모든 가능한 행동(Column)에 대해 시뮬레이션
    scores = {}
    for col in candidate_cols:
        # 가상의 행동(a) 수행 후 상태(s')의 점수 계산
        # V(s') = score_single_move(...)
        scores[col] = score_single_move(grid, col, my_mark,
                                         config)

    # 2. 가장 높은 점수를 가진 행동 선택
    best_score = max(scores.values())
    best_cols = [c for c, s in scores.items() if s ==
                  best_score]

    # 3. 최적해가 여러 개면 랜덤 선택
    return random.choice(best_cols)
```

N-Step Lookahead 알고리즘

재귀적 탐색을 통한 미래 상태 예측 및 최적 수 결정

🔗 설정 파라미터 `LOOKAHEAD_DEPTH = 3` ⚠️ 제약사항 알파-베타 가지치기 미사용으로 깊이 증가 시 연산량 급증

- **아이디어:** 가능한 열에 내 돌을 한 번 둔 뒤, LOOKAHEAD_DEPTH(기본 3)만큼 순수 미니맥스를 돌려 휴리스틱 점수를 얻음
- **복잡도:** 가지수 B와 깊이 d에 대해 $O(B^d)$. 알파-베타나 수순 정렬을 쓰지 않으므로 깊이가 커지면 비용이 급격히 증가함
- **장점:** 구현이 단순하고 결정적이며, PATTERN_SCORE 휴리스틱(내 4/3/2 연결 보너스, 상대 3/2 연결 패널티) 덕분에 짧은 전술을 빠르게 탐지함
- **단점:** 알파-베타 가지치기가 없어 깊이를 크게 늘리기 어렵고, 휴리스틱 품질 · LOOKAHEAD_DEPTH 설정에 민감함
- **주 용도:** 빠른 시뮬레이터가 있고 전술 깊이를 2~4수 정도만 확장해도 충분한 ConnectX/Connect4류 환경

N-Step Lookahead State, Action, Reward 설계

State, Action, Reward 구조 심층 분석

State (상태 표현)

게임 환경(Environment)에서 제공하는 1차원 리스트 형태의 보드 데이터를 분석하기 쉬운 (rows, columns) 형태의 2차원 배열로 변환.

Action (행동 선택)

가능한 모든 열에 대해 LOOKAHEAD_DEPTH=3까지 순수 Minimax를 재귀 호출함
내 턴에는 점수 최대화(Max), 상대 턴에는 최소화(Min) 전략을 적용하여 깊이 제한 내의 모든 경우의 수를 탐색합니다. 최고 점수를 얻은 열들 중 하나를 무작위로 선택함

가능한 action : 첫번째 열 ~ 여섯번째 열을 선택

Reward (보상 체계)

PATTERN_SCORE 딕셔너리를 통해 내 4/3/2목과 상대 3/2목에 대한 가중치를 정의함
iter_all_windows 함수로 생성된 모든 윈도우를 순회하며 점수를 누적하여,
현재 보드 상태의 윗불리를 정량적으로 평가함

핵심 포인트 요약

Recursive Search

LOOKAHEAD_DEPTH=3 설정을 통해 3수 앞까지의 미래 상태를 예측하여 의사결정

Minimax Logic

알파-베타 가지치기 없이 전체 게임 트리를 순수하게 탐색하는 Minimax 알고리즘 적용

Score System

내 4연속(+1M)과 상대 3연속(-50k) 등 공격과 수비의 가중치 균형을 계산에 반영

Exhaustive Exploration

깊이 제한 내의 모든 가능한 수를 완전히 탐색하여 해당 깊이에서의 최적해 보장

N-Step Lookahead - 탐색 깊이 설정 및 재귀적 탐색

게임 트리를 탐색하여 최적의 수를 결정하는 미니맥스 알고리즘

🔍 탐색 깊이 설정 및 재귀적 탐색

현재 상태에서 LOOKAHEAD_DEPTH (3) 만큼 재귀적으로 트리를 내려가며, 나의 턴에는 최대 이득(Max), 상대 턴에는 최소 이득(Min)을 가정하여 값을 역전파함



Maximizing Player (나)

자신에게 가장 유리한(점수가 높은) 수를 선택한다고 가정



Minimizing Player (상대)

나에게 가장 불리한(점수가 낮은) 수를 둘 것이라 가정



Leaf Node (종료 조건)

Depth=0 또는 게임 종료 시 휴리스틱 함수(evaluate)로 점수 반환

submission_n_step_lookahead.py

```
# 몇 수 앞까지 볼지 (minimax depth)
LOOKAHEAD_DEPTH = 3

def minimax(grid, cur_player, me, depth, cfg):
    """
    N-step minimax 탐색 (alpha-beta pruning 없는 단순 버전).
    """
    playable = find_playable_columns(grid.flatten(), cfg)

    # (리프 노드 도달 시 점수 반환 코드 생략) ...

    # 내 차례: 최대화
    if cur_player == me:
        best_score = -float("inf")
        for c in playable:
            next_grid = drop_token(grid, c, cur_player, cfg)
            value = minimax(next_grid, next_player, me, depth - 1, cfg)
            best_score = max(best_score, value)
        return best_score

    # 상대 차례: 최소화
    else:
        worst_score = float("inf")
        for c in playable:
            next_grid = drop_token(grid, c, cur_player, cfg)
            value = minimax(next_grid, next_player, me, depth - 1, cfg)
            worst_score = min(worst_score, value)
        return worst_score
```

N-Step Lookahead - 최종 에이전트 실행

게임 트리를 탐색하여 최적의 수를 결정하는 미니맥스 알고리즘

🏹 에이전트 실행

현재 가능한 모든 수에 대해 Minimax 값을 계산하고 가장 높은 값을 선택함

단순한 당장의 점수가 아니라, 3수 뒤의 미래 가치까지 고려된 점수를 기준으로 수행함

submission_n_step_lookahead.py

```
def my_agent(obs, cfg):  
    """미니맥스 기반 N-step lookahead agent."""  
    # (보드 변환 생략)..  
  
    scores = {}  
    for c in playable:  
        next_grid = drop_token(grid, c, me, cfg)  
        scores[c] = minimax(  
            grid=next_grid,  
            cur_player=opp,  
            me=me,  
            depth=LOOKAHEAD_DEPTH,  
            cfg=cfg  
        )  
  
    # 최고 점수를 가진 행동 선택  
    best_score = max(scores.values())  
    best_cols = [c for c, sc in scores.items() if sc ==  
best_score]  
    return random.choice(best_cols)
```

MCTS + Heuristic AlphaZero-style 알고리즘

휴리스틱 기반 위협 감지와 MCTS 탐색의 결합 구조

- **아이디어:** 정책·가치망으로 자가대국을 학습하고, 결정 시 정책 **priors**+가치로 MCTS를 가이드
- **장점:** 탐색 집중으로 강한 수, 가치망이 롤아웃 깊이 감소, 정책 **prior**로 유망 수에 집중
- **단점:** 자가대국+수마다 탐색으로 비용 높음, 정확한 시뮬레이터 필요, 커리큘럼·온폴리시 데이터 품질 중요.
- **주 용도:** 추론 시 탐색 계산을 감당하며 플레이 품질을 극대화하려는 경우

MCTS + Heuristic AlphaZero-style State,Action,Reward 설계

State, Action, Reward 구조 심층 분석

State (상태 표현)

게임 환경(Environment)에서 제공하는 1차원 리스트 형태의 보드 데이터를 분석하기 쉬운 (rows, columns) 형태의 2차원 배열로 변환.

Action (행동 우선순위)

단순 탐색 전, 치명적인 수를 먼저 필터링함:

- 0. 즉시 승리: 바로 게임을 끝낼 수 있는 수
- 1. 즉시 패배 방어: 상대의 승리를 막는 수 (안전장치)
- 2. MCTS: 위 상황이 아닐 때 몬테카를로 트리 탐색 수행

가능한 action : 첫번째 열 ~ 여섯번째 열을 선택

Reward (개선된 위협 로직)

내가 두지 않은 열(Column)까지 포함하여, 4칸 윈도우 내 상대 돌의 개수를 기반으로 승리 패턴 가능성을 정밀 검사함
단순 연속뿐만 아니라 징검다리 형태의 위협도 평가함.

Key Logic Summary

감지 가능 패턴

● 0 0 0 _ (연속 3)

● 0 0 _ 0 (2-1)

● 0 _ 0 0 (1-2)

다중 승리 루트

RETURNS 매핑 (위협 수준)

4 치명적 위협 상대가 3개를 이미 둬, 여기 두면 상대 승리

3 잠재적 위협 상대가 2개를 이미 둬 (징검다리 포함)

0 위협 없음 해당 위치는 당장 위험하지 않음

MCTS + Heuristic AlphaZero-style - 정책과 위협 감지

휴리스틱 기반 위협 감지와 MCTS 탐색의 결합 구조

🔧 핵심문제 해결

단순한 신경망 대신, 위협 감지(Gap Detection)과 휴리스틱을 결합하여 현재 상태의 가치와 각 행동의 확률을 계산함

특히 상대방의 치명적인 공격 패턴을 감지하여 높은 점수를 부여함

•하이브리드 평가: 딥러닝 모델 대신 정교한 룰 기반 평가 함수를 사용하여 Policy와 Value를 생성함

•위협 감지: `analyze_threat_window` 함수를 통해 상대방의 연속된 돌뿐만 아니라 징검다리 형태의 위협까지 감지하여 방어 점수를 크게 부여함

•Softmax 변환: 계산된 점수들을 Softmax 함수를 통과시켜 확률 분포 형태의 Policy로 변환하여 MCTS에 제공함

```
submission_alphazero_mcts_gap_defense.py

def get_policy_value(grid, player, cfg):
    # ... (유효한 행동 조회) ...
    action_scores = {}

    for action in valid_actions:
        score = 0
        new_grid = drop_piece(grid, action, player, cfg)
        # 1. 공격: 즉시 승리 (최우선)
        if check_win(new_grid, player, cfg):
            action_scores[action] = 10000000
            continue
        # 2. 방어: 윈도우 기반 위협 분석 (징검다리 포함)
        threat_level = analyze_threat_window(grid, action,
opponent, cfg)
        if threat_level == 4: score += 9000000
        elif threat_level == 3: score += 40000

        # ... (전략적 점수, 연결성 점수 합산) ...
        action_scores[action] = score

    # Softmax를 통해 행동 확률(Policy)로 변환
    # ... (Softmax 계산 및 Value 정규화) ...
    return policy, value
```


MCTS + Heuristic AlphaZero-style - 시뮬레이션을 통한 최적의 수 탐색

휴리스틱 기반 위협 감지와 MCTS 탐색의 결합 구조

🔗 핵심문제 해결

Policy와 Value를 기반으로 **MCTS(Monte Carlo Tree Search)**를 수행하여 가장 유망한 수를 찾음

UCB(Upper Confidence Bound) 공식을 사용하여 탐험(Exploration)과 활용(Exploitation)의 균형을 맞춤

•**Selection:** `ucb_score`가 가장 높은 자식 노드를 선택하여 트리를 내려갑니다. `C_PUCT` 상수로 탐험 정도를 조절함

•**Expansion:** 리프 노드에 도달하면 `get_policy_value`를 호출하여 자식 노드들을 확장하고 초기 확률(Prior)을 부여함

•**Backpropagation:** 시뮬레이션 결과(Value)를 부모 노드들로 전파하여 `visit_count`와 `value_sum`을 갱신함

submission_alphazero_mcts_gap_defense.py

```
class MCTSNode:
    # ... (초기화 생략) ...
    def ucb_score(self, child):
        if child.visit_count == 0: q_value = 0
        else: q_value = -child.value()
        u_value = C_PUCT * child.prior *
math.sqrt(self.visit_count) / (1 + child.visit_count)
        return q_value + u_value

    def expand(self):
        if is_terminal(self.grid, self.cfg): return
        policy, _ = get_policy_value(self.grid, self.player,
self.cfg)
        valid_actions =
get_valid_actions(self.grid.flatten().tolist(), self.cfg)
        for action in valid_actions:
            if action not in self.children:
                new_grid = drop_piece(self.grid, action,
self.player, self.cfg)
                prior = policy.get(action, 1.0 /
len(valid_actions))
                self.children[action] = MCTSNode(new_grid, 3 -
self.player, self.cfg, self, action, prior)
```

MCTS + Heuristic AlphaZero-style - MCTS와 안전장치의 결합

휴리스틱 기반 위협 감지와 MCTS 탐색의 결합 구조

🔗 핵심문제 해결

MCTS 탐색 결과에 전적으로 의존하지 않고, **즉시 승리**와 **즉시 패배 방어**라는 안전장치를 최우선으로 적용하여 실수를 방지한 후, MCTS를 통해 가장 많이 방문한 수를 선택함

- Rule-Based Safety:** MCTS 시뮬레이션 횟수가 부족할 경우 발생할 수 있는 치명적인 실수를 방지하기 위해, 1수 앞의 승리/패배는 룰 기반으로 즉시 처리함
- Robust Decision:** 위급한 상황이 아니라면 `mcts_search`를 통해 충분한 시뮬레이션을 거친 후, 가장 방문 횟수가 높은(Robust) 행동을 최종적으로 선택함

submission_alphazero_mcts_gap_defense.py

```
def my_agent(obs, config):
    grid = to_grid(obs.board, config)
    player = obs.mark

    # 0. 즉시 승리
    valid_actions = get_valid_actions(obs.board, config)
    for action in valid_actions:
        if check_win(drop_piece(grid, action, player, config), player,
            config):
            return action

    # 1. 즉시 패배 방어 (한번 더 안전장치)
    opponent = 3 - player
    for action in valid_actions:
        if check_win(drop_piece(grid, action, opponent, config), opponent,
            config):
            return action

    # 2. MCTS
    try:
        return int(mcts_search(grid, player, config))
    except:
        return int(random.choice(valid_actions))
```

MTD(f) 알고리즘

승리를 확정하고 패배를 원천 차단하는 계층적 의사결정 구조

- 아이디어: 추정값 f 주변의 0-차 알파-베타를 반복하며 TT로 재사용해 값 수렴.
- 장점: 순수 알파-베타 대비 노드 확장 감소, 메모리 활용 극대화, 반복적 깊이 증가와 궁합
- 단점: 초기 추정·TT 크기·수순 정렬에 민감, TT가 작거나 정렬이 나쁘면 **thrash**.
- 주 용도: 결정적 대전에서 강한 휴리스틱·수순 정렬·충분한 TT를 갖출 때.



Transposition Table (TT)

동일한 보드 상태를 해시맵에 저장하여 재방문 시 즉시 값 반환
(EXACT/LOWER/UPPER)



Iterative Deepening

얕은 깊이부터 점차 깊이를 늘려가며 탐색, 시간 초과 시에도 유효한 최선의 수 보장

MTD(f) State, Action, Reward 설계

State, Action, Reward 구조 심층 분석

State (상태 표현)

게임 환경(Environment)에서 제공하는 1차원 리스트 형태의 보드 데이터를 분석하기 쉬운 (rows, columns) 형태의 2차원 배열로 변환함

Action (행동 선택)

통합된 Negamax + MTD(f) 프레임워크를 사용함
단순히 탐색만 하는 것이 아니라, 열(column)의 높이 계산과 무의미한 수를 사전에 거르는 안전장치를 통해 탐색 속도와 안정성을 동시에 확보함

가능한 action : 첫번째 열 ~ 여섯번째 열을 선택

Reward (보상 체계)

복잡한 가중치 대신 단순화된 점수표를 사용함
상대의 3연속 공격에는 큰 벌점을 부과하고,
나의 2·3연속 공격에는 보너스를 부여하여 공수 균형이 잡힌 상대적 평가 기준을 제공함

핵심 포인트 요약

✓ Hybrid Framework

메모리 효율적인 Negamax 알고리즘과 빠른 수렴 속도의 MTD(f)를 결합하여 최적의 탐색 성능을 발휘함

✓ Transposition Table

이전 탐색 결과를 저장하고 재사용(Memoization)하여 중복 연산을 획기적으로 줄임

✓ Safety Guards

유효하지 않은 수나 필승/필패가 명확한 경로를 조기에 차단하여 탐색의 질을 높임

✓ Relative Scoring

절대적인 점수보다 상대와의 점수 차이에 집중하여, 불리한 상황에서도 최선의 방어책을 찾아냄

MTD(f) - 전략적 우선순위 기반 수 결정 (Strategic Decision Making)

승리를 확정하고 패배를 원천 차단하는 계층적 의사결정 구조

1 즉시 승리 (Immediate Win)

현재 턴에 4목을 완성할 수 있다면 즉시 착수함
높이 최적화(Height-optimized)를 통해 더 안정적인 승리 수를 선택함

2 즉시 방어 (Critical Blocking)

상대가 다음 턴에 승리할 수 있는 치명적인 위협이 있다면
무조건 차단(Forced Block)하여 게임을 지속함

3 자살수 배제 (Suicide Prevention)

내가 둔 수 바로 위에 상대가 두어 승리하는 '자살수
(Suicide Move)'를 후보군에서 사전에 필터링함

- ✓ 중앙 선호 (Center Preference): 중앙 열부터 탐색
- ✓ 무브 오더링 (Move Ordering): 유망한 수 먼저 탐색
- ✓ 반복적 심화 (Iterative Deepening): 시간 제한 내 최대 깊이

```
submission_mtdf_negamax_strategic.py

def my_agent(obs, config):
    # (초기화 생략)

    # [Step 0] Immediate Win (Attack) - 높이가 낮고 중앙에 가까운 곳 우선선
    winning_moves = []
    for col in valid_actions:
        if check_win(drop_piece(grid, col, me, config), me, config):
            winning_moves.append(col)
    if winning_moves: # 승리 가능한 수 중 높이가 낮고 중앙에 가까운 최적 수 선택
        center = config.columns // 2
        winning_moves.sort(key=lambda col: (
            get_column_height(grid, col, config), # 1순위: 높이 낮은 순
            abs(col - center)                       # 2순위: 중앙 우선
        ))
        return int(winning_moves[0])
    # [Step 1] Immediate Defense (Block) - 상대 승리를 저지
    for col in valid_actions:
        if check_win(drop_piece(grid, col, opponent, config), opponent,
            config):
            return int(col)
    # [Step 2] Filter "Suicide" Moves - 상대에게 승리를 헌납하는 수 배제
    safe_actions = []
    for col in valid_actions:
        if not check_suicide_move(grid, col, me, config):
            safe_actions.append(col)

    # (이후 탐색 진행) ...
```

MTD(f) - Negamax 알고리즘과 Move Ordering

승리를 확정하고 패배를 원천 차단하는 계층적 의사결정 구조

🔧 핵심문제 해결

Minimax 알고리즘의 간결한 형태인 **Negamax**를 사용하여 게임 트리를 탐색하며, **Transposition Table(TT)**과 **Move Ordering**을 통해 탐색 효율을 극대화함

- Negamax:** $\max(a, b) = -\min(-a, -b)$ 성질을 이용해 Max/Min 노드를 하나의 함수로 통합하여 코드를 간결하게 유지함

- Move Ordering:** 승리 가능성이 높은 중앙(Center)부터 탐색하도록 정렬하여, **Alpha-Beta Pruning**이 더 빨리, 자주 발생하도록 유도함

```
submission_mtdf_negamax_strategic.py

def negamax(grid, depth, alpha, beta, player, cfg, start_time):
    # (시간 제한 및 TT 조회 생략) ...

    # Move Ordering
    center = cfg.columns // 2
    valid_actions.sort(key=lambda x: abs(x - center))

    best_value = -float('inf')

    for col in valid_actions:
        new_grid = drop_piece(grid, col, player, cfg)
        try:
            val = -negamax(new_grid, depth - 1, -beta, -alpha, 3-player, cfg, start_time)
        except TimeoutError: raise

        if val > best_value: best_value = val
        alpha = max(alpha, best_value)

    # (TT 저장 생략) ..
    return best_value
```

MTD(f) - Null-Window Search를 통한 고속 탐색

승리를 확정하고 패배를 원천 차단하는 계층적 의사결정 구조

🔧 핵심문제 해결

Null-Window ($\beta = \alpha + 1$) 탐색을 반복하여 값에 수렴하는 방식

- **Null-Window Search:** 탐색 범위(Window)를 $[\alpha, \beta]$ 대신 $[g, g+1]$ 처럼 아주 좁게 설정하여 **negamax**를 호출하여 가지치기를 극대화하여 탐색 속도를 비약적으로 높임
- **반복 수렴:** 초기 추측값(guess)에서 시작하여, 실제 점수가 **lower**와 **upper** 사이로 좁혀질 때까지 빠르게 반복 탐색함
- **Memory-Enhanced:** 이 방식은 **Transposition Table(TT)**과 결합될 때 가장 강력한 성능을 발휘함

```
submission_mtdf_negamax_strategic.py

def mtdf_search(grid, guess, depth, player, cfg, start_time):
    g = guess
    upper = float('inf')
    lower = -float('inf')

    # 예측값이 정확한 범위 내로 수렴할 때까지 반복
    while lower < upper:
        if time.time() - start_time > TIME_LIMIT: raise
        TimeoutError
        # Null-Window 설정 [beta-1, beta]
        beta = g + 1 if g == lower else g
        # Negamax 호출
        g = negamax(grid, depth, beta - 1, beta, player, cfg,
                    start_time)
        # 탐색 결과에 따라 범위 갱신
        if g < beta: upper = g
        else: lower = g
    return g
```

02 학습 알고리즘 Learning-Based



공통 보상 설계

희소 보상 문제를 해결하고
방어적 플레이를 유도하는 리워드



DQN

CNN과 Q-Learning을 결합한
기본적인 심층 강화학습 모델



Double DQN

Selection과 Evaluation을 분리해
Q값 과대평가 문제 해결



Dueling DQN

상태 가치(V)와 이점(A)을 분리해
학습 효율과 안정성 향상

DQN (공통) 알고리즘 상세 Deep Learning

State, Action, Reward 구조 심층 분석

State (상태 표현)

1차원 리스트 형태의 보드 데이터를 (rows, columns) 2차원 배열로 변환함
CNN(Convolutional Neural Network)이 보드의 공간적 패턴과 특징을 효과적으로 학습할 수 있도록 입력 형태를 최적화함

Action (행동 선택)

CNN Q-network로 Q값을 추론함
Double DQN 공식을 적용하여 Online Network가 행동을 선택하고 Target Network가 가치를 평가함으로써 과대평가(Overestimation)를 방지함
Kaggle 제출 시에는 학습된 가중치를 NumPy EmbeddedPolicy로 내장하여 PyTorch 의존성 없이 고속 추론(argmax)을 수행함

가능한 action : 첫번째 열 ~ 여섯번째 열을 선택

Reward (보상 체계)

Kaggle 환경의 기본 리워드(+1: 승리, -1: 패배, 0: 무승부)를 그대로 사용함(Sparse Reward).
추가적인 Reward Shaping 없이 경험(Experience)을 버퍼에 적재하며,
잘못된 위치 착수 시 즉시 종료 및 음수 보상을 통해 규칙을 학습함

핵심 기술 요약

✓ Deep Learning Architecture

CNN을 활용하여 보드의 복잡한 패턴과 잠재적 위협을 시각적으로 인지하고 학습

✓ Double DQN Logic

Target Network 분리를 통해 학습의 안정성을 확보하고 Q-Value 과대평가 문제 해결

✓ Embedded Inference

NumPy 기반의 경량화된 순전파(Forward) 구현으로 외부 라이브러리 없이 Kaggle 환경 최적화

✓ Sparse Reward Handling

희소한 보상 환경에서도 Experience Replay를 통해 효율적인 샘플링과 학습 수행

보상 함수 기대 효과 및 설계 의도 Reward Design Analysis

탐색 없는 강화학습 에이전트를 위한 즉각적 피드백 시스템 설계



보상 체계 비교 분석

유형	Sparse Reward (희소)	Dense Reward (밀집)
특징	기존 방식 게임 종료 시에만 ± 1 보상 부여 (승/패)	제안 방식 매 턴마다 행동에 대한 즉각적 피드백 제공
문제점	초반 무작위 탐색 시 학습 신호 부족, 수렴 속도 저하	잘못된 보상 설계 시 최적 정책 왜곡 가능성 (Reward Hacking)
효과	정확한 승리 목표 학습 가능하나 시간이 오래 걸림	생존 보상(+1/42)을 통해 방어적 플레이와 장기전 유도

🎯 설계 의도: 탐색 부재의 보완

DQN과 같은 Value-Based RL 에이전트는 MCTS와 달리 실시간 탐색(Search)을 수행하지 않음
따라서 강제 수(Forced Move)와 같은 결정적 패턴을 스스로 학습하기 어려움
이를 보완하기 위해 규칙 위반(-10)과 생존 보상 등 강력하고 즉각적인 피드백을 통해 기본적인 게임 규칙과 전술을 빠르게 내재화하도록 설계함

🛡 방어적 플레이 유도 (Defensive Play)

ConnectX에서는 공격보다 수비 실패로 패배하는 경우가 많음
매 턴마다 부여되는 미세한 생존 보상(+1/42)은 에이전트에게 "빨리 지는 것보다 버티는 것이 이득"이라는 신호를 주어, 상대의 공격을 적극적으로 차단하는 방어적 성향을 학습하게 함



핵심 인사이트: 탐색이 없는 모델-프리(Model-Free) 알고리즘은 희소 보상 환경에서 '전술적 함정'에 취약하므로, 보상 설계(Reward Shaping)가 학습의 나침반 역할을 해야 함

DQN 타겟 계산과 과대평가 문제



Bellman Optimality Equation

DQN의 학습 목표는 현재 상태에서 선택한 행동의 Q값이 '보상 + 미래의 최대 가치'와 같아지도록 하는 것
이를 위해 타겟 네트워크를 활용함

$$Y = r + \gamma \cdot \max_{a'} Q_{\text{target}}(s', a')$$

r: 보상, γ : 할인율(0.99), s': 다음 상태, a': 다음 행동



과대평가 (Overestimation) 문제

기본 DQN은 타겟 계산 시 max 연산자를 사용함
이는 Q값의 추정 오차로 인해 특정 행동의 가치를 실제보다 높게 평가하는
편향(Bias)을 발생시키며, 학습 불안정의 원인이 됨

↗ 핵심 한계

"낙관적인 가치 추정은 에이전트가 차선의 정책으로 수렴하게 만들"
→ Double DQN 도입의 직접적인 배경

DQN 학습 루프 Training Loop

강화학습 에이전트의 순차적 학습 프로세스

1



행동 선택

Epsilon-Greedy 정책으로 탐험 (Random)과 활용(Max Q) 균형 조절

```
# Epsilon Decay
epsilon =
epsilon_by_step(step)
if random.random() < epsilon:
action = random.choice()
else:
# Greedy Action
action = q_net(state).argmax()
```

2



경험 저장 & 샘플링

전이(s,a,r,s')를 버퍼에 저장하고 무작위 배치 샘플링으로 학습

```
# Store Transition
buffer.push(s, a, r, next_s,
done)
if len(buffer) ≥ BATCH_SIZE:
# Random Sampling
batch = buffer.sample(BATCH)
optimize_dqn(q_net, batch)
```

3



손실 계산 & 역전파

TD Error(예측값 - 타겟값)를 최소화하도록 네트워크 업데이트

```
# Calculate Loss (MSE)
loss = F.mse_loss(q_values,
targets)
# Backpropagation
optimizer.zero_grad()
loss.backward()
optimizer.step()
```

4



타겟 동기화

학습 안정화를 위해 일정 주기마다 Target Network 가중치 업데이트

```
# Hard Update
if step % TARGET_SYNC == 0:
target_net.load_state_dict(
q_net.state_dict()
)
# Stabilizes Training
```

DQN: 네트워크 구조

네트워크 구조 (CNN Architecture)

✓ 핵심 문제 해결 1

특징: 6×7 보드판의 시각적 패턴을 인식하기 위해 합성곱 신경망(CNN) 사용.

구조: Conv2d(32) → Conv2d(64) → Flatten → Linear(Action 7).

submission_drl_dqn.py

```
class QNetwork(nn.Module if HAS_TORCH else object):
    def __init__(self, action_dim: int = 7):
        # (초기화 생략) ...
        self.cnn = nn.Sequential(
            # 1. 특징 추출 (Feature Extraction)
            nn.Conv2d(1, 32, kernel_size=3, stride=1),
            nn.ReLU(),
            nn.Conv2d(32, 64, kernel_size=3, stride=1),
            nn.ReLU(),
            nn.Flatten(),
        )
        # 2. 행동 가치 추정 (Q-Value Estimation)
        self.fc = nn.Linear(64 * 2 * 3, action_dim)
```

DQN: 타겟 계산

DQN 타겟 계산 (Bellman Optimality Equation)

✓ 핵심 문제 해결 2

핵심: 다음 상태(s')에서 가장 높은 Q값(max)을 직접 사용하여 타겟을 설정함.

수식: $Y = r + \gamma \max_{a'} Q_{\text{target}}(s', a')$

한계: 특정 행동의 가치를 과대평가(Overestimation)하는 경향이 있음 (Double DQN 도입의 계기).

```
submission_drl_dqn.py

def optimize_dqn(q_net: QNetwork, target_net: QNetwork, batch:
Transition, optimizer: torch.optim.Optimizer,) → float:

    # ... (데이터 로딩 생략) ...

    with torch.no_grad():
        # Target Network에서 다음 상태의 최대 Q값(Max Q) 계산
        max_next = target_net(next_states_t).max(dim=1,
keepdim=True)[0]

        # 벨만 방정식(Bellman Equation)에 따른 타겟값 설정
        targets = rewards_t + GAMMA * max_next * masks_t

    # 예측값(Q)과 타겟값(Y)의 차이(MSE)로 학습
    loss = F.mse_loss(q_values, targets)
    # ... (역전파) ...
```

DQN: 학습 루프

학습 루프 (Training Loop)

✓ 핵심 문제 해결 3

Epsilon-Greedy: 초기에는 무작위 탐색($\epsilon=0.1$)을 하다가 점차 최적 행동 선택 비율을 높임.

Target Sync: 학습 안정화를 위해 일정 주기(TARGET_SYNC)마다 Target Network를 업데이트함.

submission_drl_dqn.py

```
def train_dqn_agent(episodes: int = 3000) → QNetwork:
    # ... (생략) ...
    for episode in range(episodes):
        # 탐험(Exploration) 비율 감소 (Decay)
        epsilon = epsilon_by_step(global_step)

        # ... (행동 선택 및 환경 상호작용) ...

        # Experience Replay: 버퍼에서 샘플링하여 학습
        if len(buffer) ≥ max(BATCH_SIZE, MIN_REPLAY):
            batch = buffer.sample(BATCH_SIZE)
            optimize_dqn(q_net, target_net, batch, optimizer)

        # Target Network 동기화 (Soft/Hard Update)
        if (episode + 1) % TARGET_SYNC == 0:
            target_net.load_state_dict(q_net.state_dict())
```

Double DQN: 타겟 계산 개선 Overestimation 해결

행동 선택(Selection)과 가치 평가(Evaluation)의 분리

✓ 핵심 문제 해결 1

기본 DQN은 최댓값(max) 연산으로 인해 Q값을 과대평가하는 경향이 있음
Double DQN은 두 개의 네트워크를 사용하여 이 편향을 제거함.

1. 행동 선택 (Selection)

Online Network(θ)가 최적의 행동을 선택

2. 가치 평가 (Evaluation)

Target Network(θ')가 선택된 행동의 가치를 평가

$$Y = R + \gamma \cdot Q'(s', \operatorname{argmax} Q(s', a; \theta); \theta')$$

```
submission_drl_double.py

def optimize_double_dqn(q_net: QNetwork, target_net: QNetwork, batch:
Transition, optimizer: torch.optim.Optimizer):
    # ... (전처리 코드 생략) ...

    # 타겟 값 계산
    with torch.no_grad():
        # 행동 선택 (Action Selection) : Online Network 사용
        next_q_online = q_net(next_states_t)
        next_actions = next_q_online.argmax(dim=1, keepdim=True)
        # 가치 평가 (Value Evaluation) : Target Network 사용
        next_q_target = target_net(next_states_t)
        # Target Net에서 Online Net이 선택한 행동의 Q값 가져옴.
        next_values = next_q_target.gather(1, next_actions)
        # 벨만 방정식 적용 (TD Target)
        targets = rewards_t + GAMMA * next_values * masks_t

    loss = F.mse_loss(q_values, targets)
    # ... (역전파 코드 생략) ...
```


Double DQN: CNN 네트워크 구조 (Neural Network Architecture)

보드 상태를 이미지처럼 인식하여 특징을 추출함

✓ 핵심 문제 해결 2

2개의 Conv Layer를 통해 가로/세로/대각선 패턴을 학습함.

입력: (1, 6, 7) 형태의 보드 상태

출력: 7개의 열(Column) 각각에 대한 Q-Value

특징 추출: 3x3 필터를 사용하여 4목 게임의 핵심인 연속된 돌의 패턴을 파악함

submission_drl_double.py

```
class QNetwork(nn.Module if HAS_TORCH else object):
    def __init__(self, action_dim: int = 7):
        # ... (초기화 생략) ...
        self.cnn = nn.Sequential(
            # 첫 번째 합성곱 층: 채널 1 → 32, 커널 3x3
            nn.Conv2d(1, 32, kernel_size=3, stride=1),
            nn.ReLU(), # 활성화 함수
            # 두 번째 합성곱 층: 채널 32 → 64, 커널 3x3
            nn.Conv2d(32, 64, kernel_size=3, stride=1),
            nn.ReLU(),
            nn.Flatten(), # 1차원 벡터로 변환
        )
        # 완전 연결 층 (Fully Connected): 행동 개수(7개)만큼 출력
        self.fc = nn.Linear(64 * 2 * 3, action_dim)

    def forward(self, x: torch.Tensor) → torch.Tensor:
        # CNN → FC 순전파
        return self.fc(self.cnn(x))
```

Double DQN: 추론 전용 정책 (Embedded Policy for Kaggle)

학습된 PyTorch 가중치를 NumPy로 변환하여 제출 환경에서 실행함

✓ 핵심 문제 해결 3

- Kaggle 환경 제약(라이브러리 로딩 시간, 파일 크기 등)을 극복하기 위해 구현함
- torch.nn.Conv2d 등의 연산을 self._conv 메서드와 np 연산으로 직접 구현하여 이식성을 높임

submission_drl_double.py

```
class EmbeddedPolicy:
    def __init__(self, literal: Dict[str, List]):
        # 학습된 가중치(Dictionary)를 NumPy 배열로 로드
        self.w1 = np.asarray(literal["cnn.0.weight"],
                               dtype=np.float32)
        # ... (나머지 가중치 로드 생략) ...

    def act(self, state: np.ndarray, playable: List[int]) →
    int:
        # NumPy 연산만으로 CNN 순전파(Forward) 구현
        x = state.astype(np.float32)
        x = self._conv(x, self.w1, self.b1) # 수동 Conv 연산
        x = np.maximum(x, 0, out=x)         # ReLU
        # ... (중략) ...
        logits = self.fc_w @ x.reshape(-1) + self.fc_b

        # 불가능한 행동 마스킹 후 최적 행동 선택
        masked = np.full_like(logits, -np.inf)
        masked[playable] = logits[playable]
        return int(np.argmax(masked))
```

Dueling DQN: 네트워크 구조 (Target Calculation)

하나의 신경망 내부를 두 개의 줄기(Stream)로 분리하는 구조적 특징

✓ 핵심 문제 해결 1

CNN으로 특징을 추출한 후,
상태 가치(V)와 행동 이점(A)을 각각 별도로 계산함.

기존 DQN은 Q값 하나만 출력하지만,
Dueling DQN은 V(s)와 A(s,a)로 역할을 분담함.

행동 간의 차이가 미미한 상태에서도
V(s) 학습을 통해 효율적인 업데이트가 가능함.

submission_drl Dueling.py

```
class DuelingQNetwork(nn.Module if HAS_TORCH else object):
    def __init__(self, action_dim: int = 7):
        # ... (CNN 층 정의 생략) ...

        # Stream 1: State Value (V) - 상태 자체의 좋고 나쁨 평가
        # (출력 1개)
        self.fc_value = nn.Linear(64 * 2 * 3, 128)
        self.value = nn.Linear(128, 1)

        # Stream 2: Action Advantage (A) - 각 행동의 상대적 우위
        # 평가 (출력 7개)
        self.fc_adv = nn.Linear(64 * 2 * 3, 128)
        self.adv = nn.Linear(128, action_dim)
```

Dueling DQN: 결합 층

분리된 V와 A를 다시 합쳐 최종 Q값을 도출하는 과정

✓ 핵심 문제 해결 2

평균을 빼주는 연산(Mean Subtraction)을 통해 식별성(Identifiability) 문제를 해결함.

단순히 $V + A$ 를 하면 V와 A의 값이 유일하게 결정되지 않음.

A의 평균을 0으로 만들어 V가 진정한 상태 가치를 나타내도록 강제함.

submission_drl_double.py

```
def forward(self, x: torch.Tensor) → torch.Tensor: #
type: ignore[override]
    feat = self.cnn(x)
    v = F.relu(self.fc_value(feat))
    a = F.relu(self.fc_adv(feat))
    v = self.value(v) # 스칼라 값 (Batch, 1)
    a = self.adv(a)   # 벡터 값 (Batch, 7)

    # 결합 공식:  $Q(s,a) = V(s) + (A(s,a) - \text{mean}(A(s,a)))$ 
    return v + a - a.mean(dim=1, keepdim=True)
```

Dueling DQN: Double DQN 방식 적용

Dueling 구조를 사용하더라도 학습 방법은 Double DQN 방식을 채택하여 안정성을 높임

✓ 핵심 문제 해결 3

핵심 포인트: Dueling 아키텍처 위에서 Double DQN의 오차 계산 방식을 그대로 사용함.

구조는 Dueling, 학습법은 Double DQN을 결합한 형태임(Dueling Double DQN).

과대평가 방지(Double)와 학습 효율성 증대(Dueling) 효과를 동시에 얻음.

submission_drl_double.py

```
def optimize_double_dueling(
    q_net: DuelingQNetwork,
    target_net: DuelingQNetwork,
    batch: Transition,
    optimizer: torch.optim.Optimizer,
) → float:

    # ... (중략) ...
    with torch.no_grad():
        # Dueling Net으로 행동 선택 (Selection)
        next_q_online = q_net(next_states_t)
        next_actions = next_q_online.argmax(dim=1, keepdim=True)

        # Target Net으로 가치 평가 (Evaluation)
        next_values = next_q_target.gather(1, next_actions)
        targets = rewards_t + GAMMA * next_values * masks_t
```

03 실험 및 결과 Results & Analysis

탐색과 학습 알고리즘의
성능을 검증하고 비교하는
데이터 기반 분석



승률 비교

선공/후공 조건에서의
알고리즘별 승리 확률 분석



강제수 대응률

즉각적인 승리/차단 상황에서의
대응 성공률 정량 평가



비용 vs 성능

의사결정 시간과 탐색 노드 수 대비
성능 트레이드오프 분석

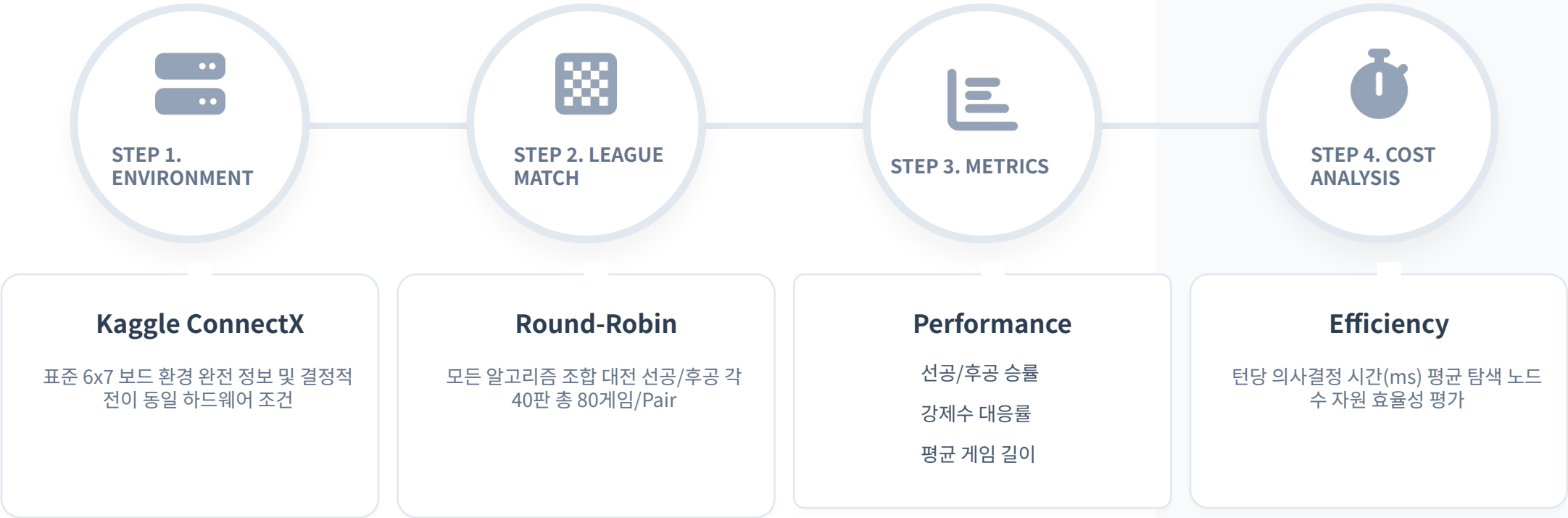


DQN 비교 분석

학습 기반 에이전트 간의
상대적 성능 및 한계점 고찰

실험 설계 및 방법론 Experimental Setup

공정한 비교를 위한 체계적인 대전 환경 및 평가지표



신뢰도 확보 전략: DQN 등 학습 기반 에이전트는 초기화 시드(Seed)를 변경하며 반복 실험하여 통계적 변동성을 최소화하고, 평균 성능과 표준편차를 함께 보고함.

DQN 알고리즘 비교 및 최적 모델 도출

Double DQN을 최종 모델로 채택한 근거

🔍 평균 턴 수: 안정적인 경기 운영

최장 플레이 타임: Double 모델이 포함된 매치업에서 평균 25턴으로 가장 긴 게임 턴 수를 기록함.

높은 방어력: 턴 수가 길다는 것은 공격에 무너지지 않고 경기를 이끌어가는 생존 능력을 보여줌.

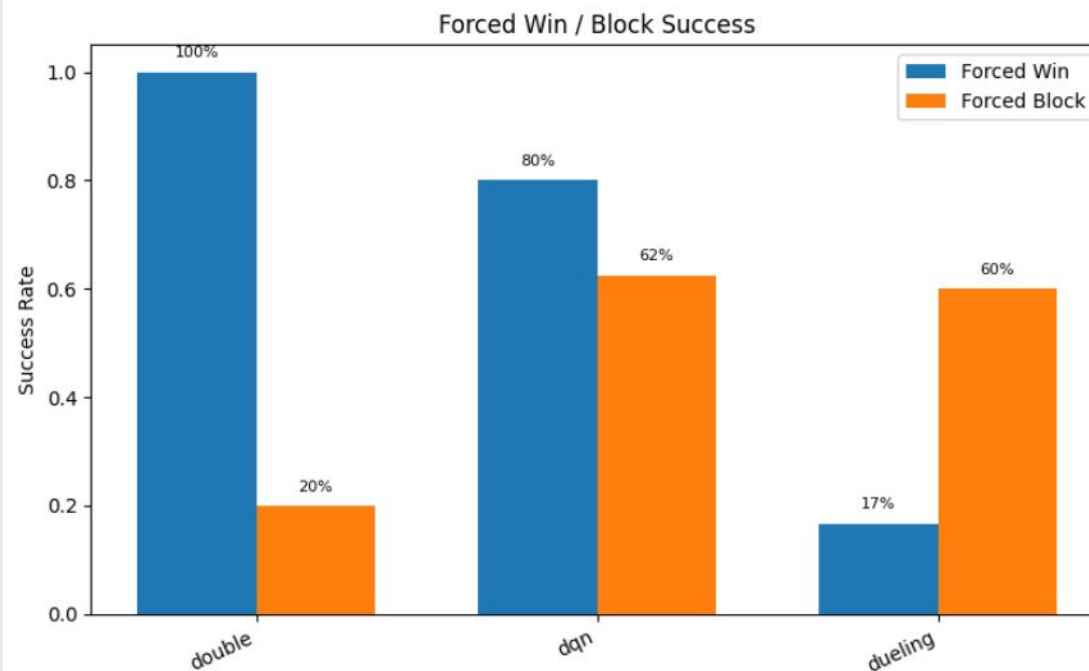
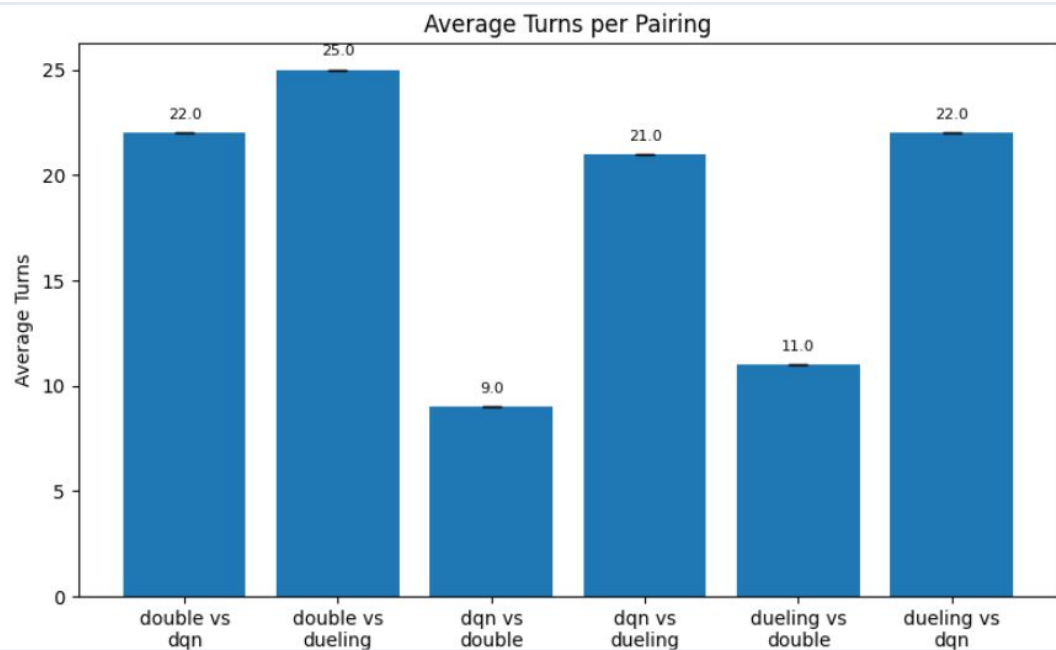
지속 가능한 퍼포먼스: 장기적인 수 싸움에서 유리한 고지를 점할 수 있다고 판단하여 채택함.

⚠️ 승리/방어 성공률: 완벽한 결정력

압도적인 승리 확정률: 필승 시나리오에서 타 모델(80%, 17%)을 제치고 유일하게 100%의 성공률을 달성함.

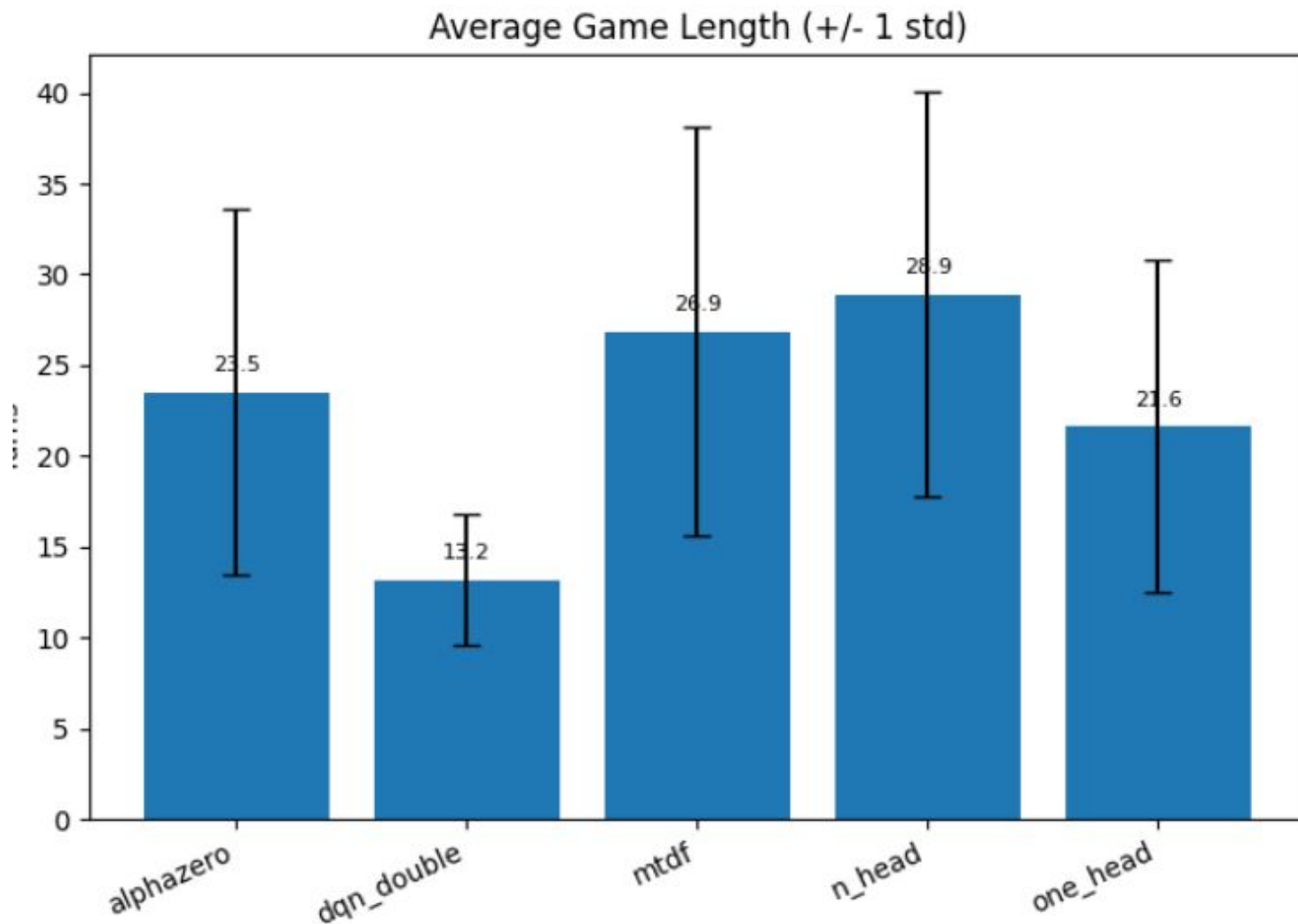
확실한 마무리 능력: 승리 기회가 포착되었을 때, 실책 없이 게임을 끝내는 신뢰도 높은 결정력을 입증함.

공격 중심의 최적화: 방어 성공률은 다소 낮으나, 기회를 100% 살리는 공격 효율성을 높게 평가하여 최종 선정함.



알고리즘별 평균 게임 길이 분석

알고리즘별 평균 종료 턴(Turns) 비교 및 안정성 평가



👑 MTD(f)의 높은 변동성

평균 26.9수의 긴 호흡과 넓은 표준편차를 보이며, 정해진 패턴이 아닌 상황에 따라 유연하게 대처함. 다양한 승리 시나리오를 탐색하며 게임을 주도적으로 이끌어 나감

🔍 N-head의 장기전 운영

평균 28.9수로 가장 긴 승부를 펼치며, 상대의 공격을 끝까지 방어해내는 끈기 있는 플레이 스타일을 보여줌. 단순한 패턴 매칭을 넘어 복잡한 수 싸움을 유도하는 심층적인 전략성을 확인함.

⚠️ DQN(Double)의 빠른 전개

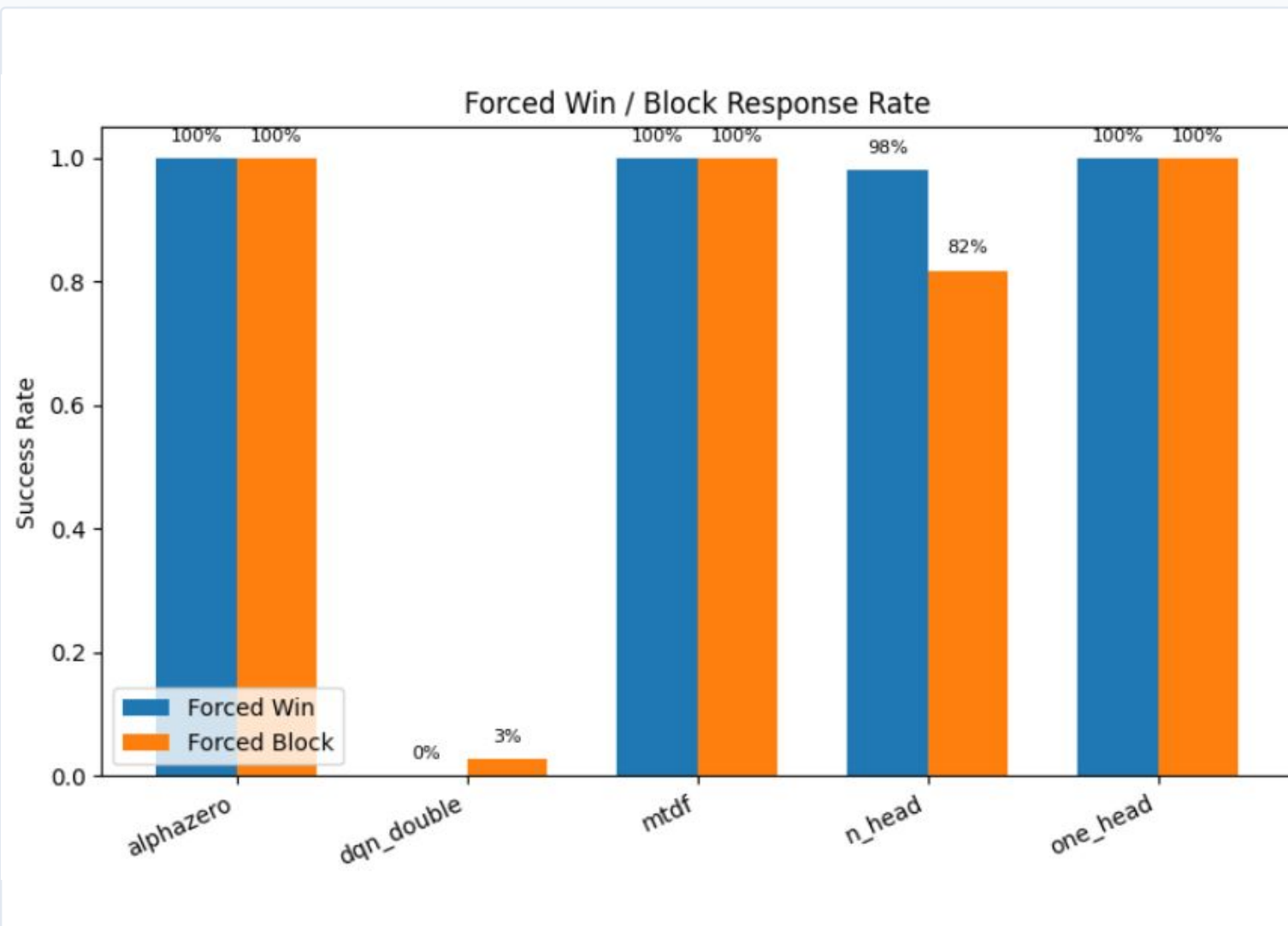
평균 13.2수로 게임을 빠르게 전개하는 속도감 있는 특성을 보임. 단위 시간당 더 많은 에피소드를 소화할 수 있어, 초기 학습 단계에서 빠른 피드백을 통한 모델 개선 가능성을 보여줌.

🏆 AlphaZero, One-head의 효율적 밸런스

AlphaZero(23.5수)와 One-head(21.6수)는 공수 밸런스가 잘 잡힌 안정적인 게임 길이를 유지함. 불필요한 장기전을 피하고 최적의 경로를 탐색하는 효율적인 의사결정 프로세스를 갖춘 것으로 평가됨.

강제수 대응률 분석

알고리즘별 강제 승리(Forced Win) 및 강제 차단(Forced Block) 성공률



✓ 전술적 완벽성

100% Success

AlphaZero, MTD(f), One-head는 강제 승리와 강제 차단 테스트에서 모두 **100%** 성공률을 달성함. 위기 상황 감지와 승리 기회 포착에 있어 전술적 완성도를 갖추었음을 입증함.

! N-Head의 역설

Block 82%

N-head는 **98%**의 높은 강제 승리 확률을 보였으나, 강제 차단율은 **82%**로 비교군 대비 다소 낮게 나타남. 공격적인 수 읽기에 집중하느라 상대의 결정적인 위협인 강제수를 일부 놓치는 경향을 확인함.

✗ DQN 전술 부재

Block 3%

Double DQN은 강제 승리 **0%**, 강제 차단 **3%**로 유의미한 전술적 대응을 보여주지 못함. 별도의 탐색 과정 없이 신경망의 가치 판단만으로는 4목 게임의 치명적인 수 읽기 패턴을 학습하기 어려움.

에이전트 별 선공 후공 비교

| 알고리즘 에이전트 별 선공 후공에 따른 결과 비교

	선공 승리	승률	후공 승리	승률
One step	21	53%	19	48%
N step	24	60%	29	73%
MTD(f)	29	73%	39	98%
Alphazero	21	53%	17	43%
DQN_double	0	0%	0	0%

⚡ MTD(f): 후공 강자

Transposition Table과 전략전 우선 순위 결합을 통해 선공의 이점을 무력화하고 오히려 후공에서 가장 안정적인 성능

⌂ N-Step

깊이 있는 탐색을 시도했으나 효율적인 방어 규칙이 부족하여 성공/후공의 강제수에 완벽히 대응하지 못한 한계

🏰 AlphaZero-style / One step

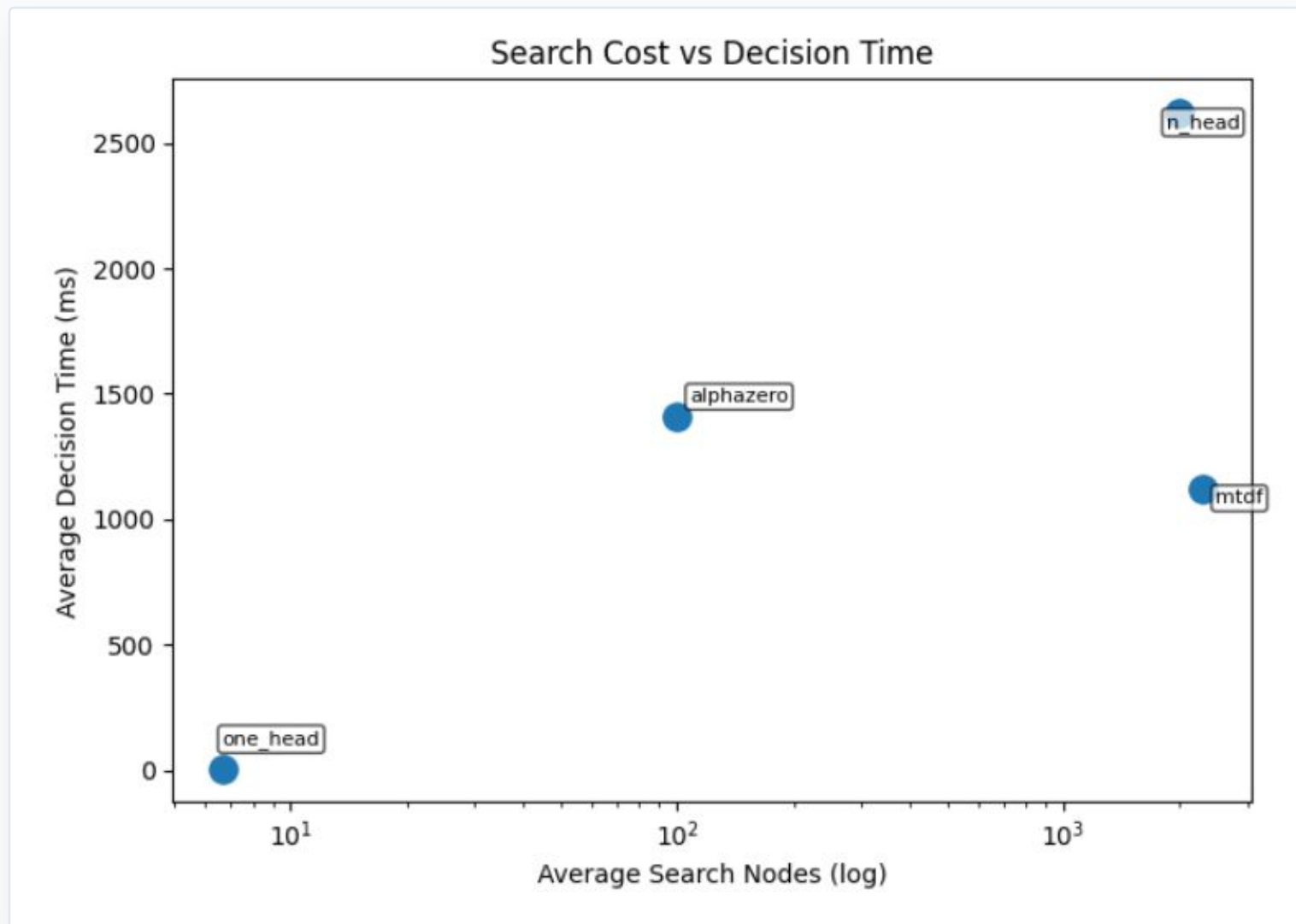
휴리스틱 기반 MCTS를 통해 유망한 경로만 선별적으로 탐색하는 Zlphazero 스타일 에이전트와 한수만 바라보는 One Step 에이전트는 중간 수준의 성능을 보임

🕒 DQN

탐색이 배제되어 있는 순수 모델이기 때문에 강제수 학습에 실패하여 탐색 기반 에이전트 상대로 전패 함.

탐색 비용 비교

| 알고리즘별 의사결정 시간(ms) 및 탐색 효율성 분석



🕒 N-Step: 가장 느린 탐색 2,500ms 이상

높은 연산 비용: 평균 의사결정 시간이 오래 걸리며, 탐색 노드 수 또한 최상위권에 위치함.
비효율적 탐색: 방대한 양의 노드를 방문하고 연산 시간도 많이 소요되어, 실시간성이 중요한 상황에서는 부적합한 모델임.

⚡ MTD(f): 높은 효율성 1,000 ~ 1,300ms

압도적인 노드 처리량: AlphaZero보다 훨씬 많은 노드를 탐색함에도 불구하고, 소요 시간은 오히려 더 짧음
최적화된 알고리즘: 단위 노드당 연산 비용이 매우 낮아, 깊은 탐색(Deep Search)을 수행하면서 빠른 응답 속도를 유지함.

🏠 AlphaZero-style 1,400 ~ 1,500ms

신경망 기반 탐색: 무거운 신경망 연산으로 인해 시간은 MTD(f)보다 느리게 소요됨.
탐색의 질적 집중: 많은 수를 내다보기보다, Policy/Value Network를 통해 유망한 경로만 선별적으로 탐색함.

🕒 즉각적 추론 (One-head) 0 ~ 5ms

Zero Search Cost: 탐색 노드 수와 소요 시간(~5ms)이 거의 0에 수렴하는 극단적인 반응 속도를 보임.
직관적 실행: 별도의 트리 탐색(Tree Search) 과정 없이, 학습된 신경망의 직관만으로 즉시 행동을 결정함.

Seed Reliability 검증

모델 안정성 및 신뢰도 분석
시드별 승률 변동성 및 95% 신뢰구간(CI) 비교

검증 방법론

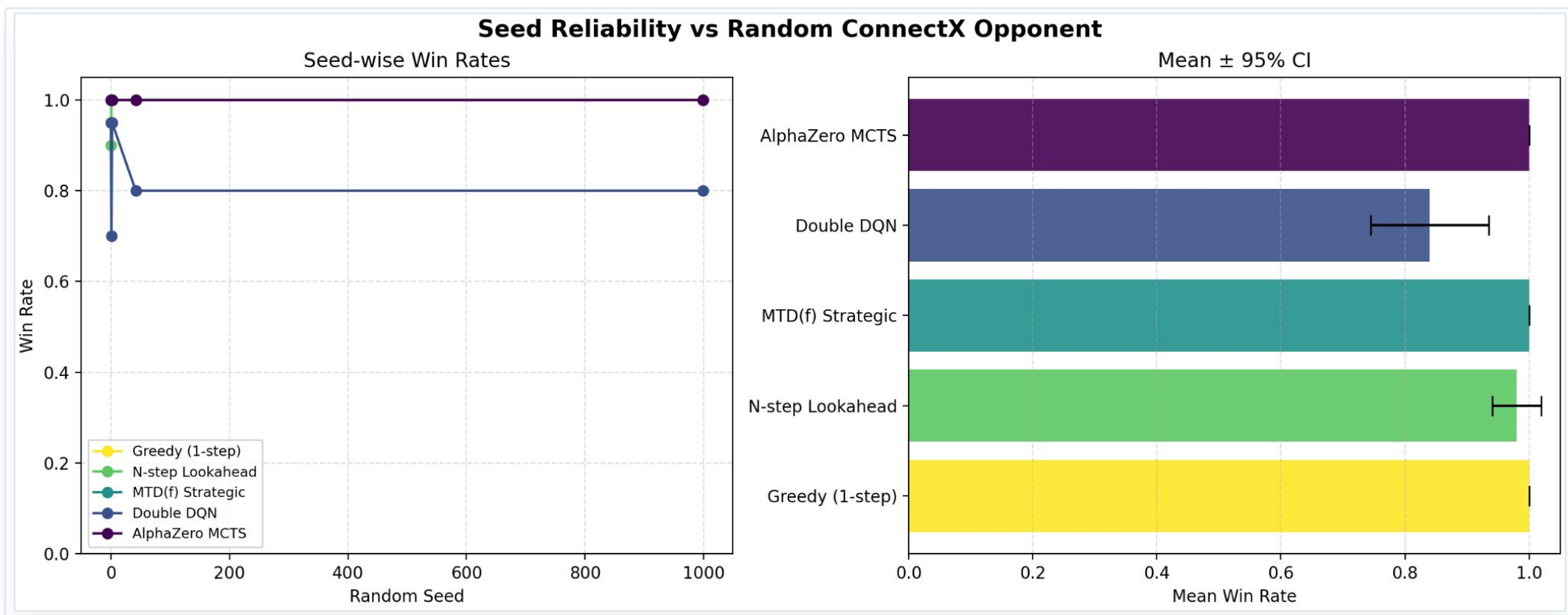
실험 설계: 주요 5개 에이전트를 대상으로 5가지 시드(0, 1, 2, 42, 999)에서 각 20회 대전을 수행하여 승률 궤적과 신뢰구간을 도출함.

결과 해석: 시드별 승률 변화(좌측 그래프)와 평균 승률의 에러바(우측 그래프)를 통해 특정 난수 환경에서의 성능 급락 여부를 확인함.

모델별 안정성 평가

완벽한 안정성: Greedy, MTD(f), AlphaZero 모델은 모든 시드에서 승률 1.0을 유지하며 평평한 궤적과 에러바 없는 완벽한 신뢰도를 입증함.

강화학습 모델의 특성 확인: Double DQN은 평균 80% 후반대의 승률을 기록했으나, 시드에 따라 신뢰구간이 넓게 나타나는 분산(Variance) 특성을 확인함.



DQN 변형 비교 분석

학습 기반 알고리즘의 아키텍처별 성능 및 특성 비교



Basic DQN

BASELINE

강제 승리 (Forced Win)80%

강제 차단 (Forced Block)62.5%

! 특징:

공격과 수비 밸런스는 존재하나, 과대평가 (Overestimation) 편향으로 인해 전체적인 학습 안정성이 낮아 최종 승률은 가장 저조함.



Double DQN

HIGH AGGRESSION

강제 승리 (Forced Win)100%

강제 차단 (Forced Block)20%

✓ 핵심 강점:

행동 선택 분리로 인해 공격 패턴(킬각)을 100% 포착하는 정확성을 보임. 단, 수비보다는 공격 위주의 'Glass Cannon' 성향을 띠.



Dueling DQN

EFFICIENCY

강제 승리 (Forced Win)17%

강제 차단 (Forced Block)60%

🔍 분석:

상태 가치(V)를 분리하여 학습 효율은 높으나, 이번 실험에서는 공격적인 수읽기보다 수동적인 플레이를 보여 상대적으로 낮은 결정력을 기록함.

💡 **종합 결론:** Double DQN이 Dueling DQN과의 맞대결에서 5전 전승 을 거두며, 공격적인 전술 패턴 학습에서 구조적 우위를 입증함

종합 분석: 핵심 발견사항

탐색 알고리즘과 학습 모델의 성능 비교 요약



Dominant Strategy

MTD(f)의 압도적 우위

97.5%

후공 승률 97.5% 달성.
Transposition Table과 전략적 우선순위(즉시 승리/방어)
결합으로 가장 안정적인 성능을 증명함



The Paradox

N-step의 깊이와 한계

81.8%

One-step 대비 승률은 높지만(60%),
명시적 패턴 체크 부재로 강제 차단률(81.8%)은
오히려 떨어지는 '깊이의 역설'을 확인함



Efficiency

비용 vs 성능 균형

1.1s

MTD(f)는 N-step(2.6s)보다 2배 이상 빠르면서도 더 높은
승률을 기록하며,
효율적인 가지치기의 중요성을 입증함



Learning Gap

0% 탐색 부재의 한계 (DQN)

탐색형 에이전트 상대로 전패(0승) 및 강제 차단률 2.6% 기록.
순수 학습 모델은 Lookahead 없는 상황에서 결정적인 전술 패턴(Trap)을 학습하기 어렵다는
한계가 명확함



Winning Formula

Rule+Depth 승리의 조건

단순한 '깊은 탐색'보다는 '명시적 규칙(Rule) + 효율적 탐색(Depth)'의 결합이 승률을 결정함
학습 모델도 최소한의 전술적 룩어헤드가 결합되어야 경쟁력을 가질 수 있음을 시사함

결론 및 고찰

👑 핵심 결론 (Core Finding)

완전정보 게임 ConnectX 환경에서 **MTD(f)** 탐색 알고리즘이 **후공 승률 97.5%**와 **강제수 대응 100%**를 달성하며 가장 우수한 성능을 보였음

반면, 탐색이 배제된 순수 **DQN** 학습 모델은 **결정적인 전술 패턴(강제수) 학습에 한계**를 보였음

주요 시사점

탐색의 중요성: 깊이 있는 탐색(N-step, MTD(f))이 즉각적인 패턴 반응(One-step)보다 장기적 승률 확보에 필수적임

하이브리드 가능성: AlphaZero-style(MCTS+Heuristic)은 적은 시뮬레이션으로도 안정적인 방어력을 보여, 탐색과 학습의 결합 가능성을 시사함

DQN의 한계: 희소 보상 환경에서 순수 RL은 '강제 패배' 패턴을 피하는 규칙을 스스로 학습하는 데 오랜 시간이 걸림



이론적 기여 (Theoretical)

강제수 대응률(Forced Move Rate)을 새로운 성능 지표로 도입하여, 단순 승률이 설명하지 못하는 에이전트의 전술적 완성도를 정량화함



실무적 기여 (Practical)

EmbeddedPolicy 클래스를 구현하여, PyTorch 모델의 경량화 및 의존성 없는 배포 파이프라인 구축



향후 과제 (Future Work)

DQN 학습에 커리큘럼 러닝(강제 패턴 우선 학습) 도입 및 MCTS 시뮬레이션 횟수 확대를 통한 성능 개선 연구 필요

한계 및 향후 계획 Limitations & Future Work

! 연구의 한계점

DQN 학습 제한: 제한된 에피소드 수로 인해 학습이 완전히 수렴하지 못해 강제수 학습에 실패

샘플 크기 부족: 알고리즘 간 비교 실험이 각 80게임(DQN 30게임)으로 통계적 유의성 확보에 제한

AlphaZero 구현 차이: 표준 AlphaZero 대신 휴리스틱 기반 MCTS를 사용하여 완전한 성능 비교에 한계

🚀 향후 개선 계획

탐색 알고리즘 고도화: N-Step에 Alpha-Beta Pruning 및 Quiescence Search 적용하여 효율성 증대

DQN 커리큘럼 학습: 강제 승리/방어 패턴을 우선 학습하는 커리큘럼 도입으로 초기 성능 개선